ENGINEERING PORTFOLIO

Nishit Kumar

Associate Technical Consultant · Full Stack · Data · AI

Perficient Inc. — Product Development (Custom Delivery) · Bengaluru, India

Computer Science & Engineering graduate and software developer, currently an Associate Technical Consultant at Perficient. Full-stack applications built end to end — React and Angular front ends, Java Spring Boot and Python services, REST APIs and relational databases — with applied work in data science, machine learning and generative AI.

---

email Nishit.kumar@perficient.com

linkedin linkedin.com/in/nishit-kumar-77a3aa354

github github.com/Nishitkumar25

website nishitkumar25.github.io

generated from nishitkumar25.github.io — content identical to the live site

/01 / Professional Profile

I am a Computer Science & Engineering graduate from Amrita Vishwa Vidyapeetham, Bengaluru, working as an **Associate Technical Consultant at Perficient Inc.** in the Product Development (Custom Delivery) business area. I work in a professional technology environment and am building practical software engineering experience across the full delivery cycle.

My engineering background is full stack. On the frontend I work with React and Angular; on the backend with Java and Spring Boot, Node.js and Python. I design and integrate REST APIs, handle authentication and authorization, model and manage relational and document databases, and test and debug what I build rather than assuming it works. Alongside that I work with Python for data science and machine learning, and with AI and generative AI.

My capstone work covers three substantial applications: a full-stack e-commerce platform, an enterprise account-onboarding system for wealth management, and an advisor-facing portfolio and risk dashboard. My professional interests are software engineering, enterprise application development, data science, AI and full stack development.

focus_areas

01	full stack development
02	frontend · backend engineering
03	rest api development
04	database management
05	data science · ml
06	ai · generative ai
07	testing · debugging

build · integrate · test · explain

/02 / Technical Skills

Technologies I work with across frontend, backend, data and tooling. Grouped by area rather than scored — depth varies by technology and is best judged from the projects below.

programming_languages

Python · Java · JavaScript · C

frontend_development

HTML5 · CSS3 · React.js · Angular · Tailwind CSS · Bootstrap · Vite

backend_development

Java Spring Boot · Node.js · Express.js · Python Flask · REST API Development · Spring Security · JWT Authentication

databases

PostgreSQL · MySQL · MongoDB

data_science_&_ml

Pandas | NumPy | Scikit-learn | XGBoost | LightGBM | Data Analysis | Machine Learning

tools_&_platforms

Git | GitHub | VS Code | Postman | Swagger / OpenAPI | Jupyter Notebook | XAMPP

other_technical_areas

Full Stack Development | API Integration | Database Design | Authentication & Authorization | Software Testing | Debugging | AI & Generative AI

— /03 / Selected Projects

Three major capstone platforms built at Perficient, followed by earlier internship, research and hackathon work. Every technology and feature listed below is taken from the actual implementation.

capstone_platforms / 03 major projects

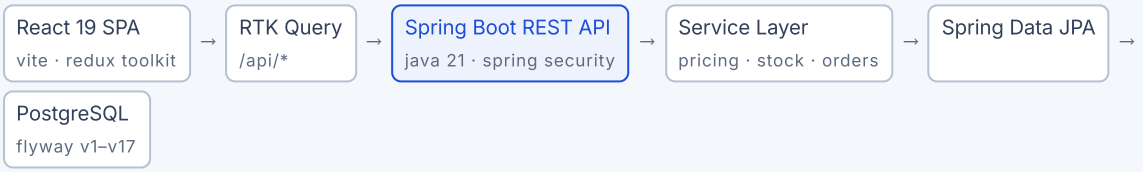
capstone_04 / perficient

FULL-STACK DEVELOPMENT · E-COMMERCE · SPRING BOOT · REACT

PawPantry — Full Stack Pet Supplies E-commerce Platform

A full-stack e-commerce application that lets customers explore and purchase pet supplies and manage the shopping and service experience around it — catalogue and search, reviews, cart and wishlist, checkout, order tracking and repeat-delivery subscriptions — with a React storefront on a Spring Boot REST API where the server is the single source of truth for prices, stock, roles and orders.

architecture full_stack_integration



technology

React 19 · Vite 8 · React Router 6 · Redux Toolkit with RTK Query · Java 21 · Spring Boot 3.5 · Spring MVC · Spring Data JPA · Spring Security with JWT · PostgreSQL with Flyway · OpenAPI 3 · Postman · JUnit 5 · Vitest · Playwright · Git & GitHub

key features

Registration and JWT authentication · product catalogue and admin product management · server-side search, filtering and sorting · reviews and ratings · cart and wishlist with guest-cart merge · three-step checkout and order management · order tracking with PDF invoice · Subscribe & Save repeat deliveries · AI-assisted natural-language catalogue search

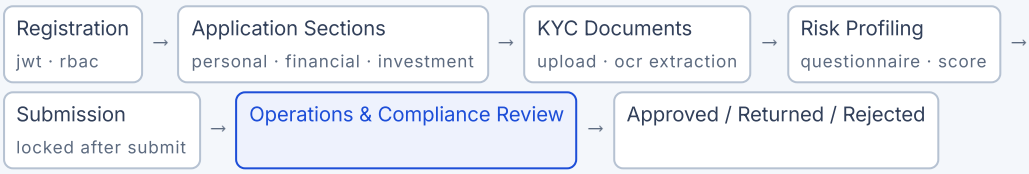
capstone_06 / perficient

ENTERPRISE APPLICATION · FINTECH · BFSI · SPRING BOOT

Wealth Management — New Account Opening Platform

A digital account onboarding solution that streamlines opening wealth management and investment accounts. A client registers, completes a multi-section application covering personal, financial, investment and regulatory information, uploads KYC documents and submits; Operations and Compliance then review the application and approve, return or reject it — every state change recorded in an append-only history.

workflow onboarding_lifecycle



technology

Java 25 · Spring Boot 4.1 · Spring MVC · Spring Data JPA / Hibernate · Spring Security (OAuth2 resource server, JWT + rotating refresh tokens, BCrypt) · PostgreSQL with Flyway · Tesseract OCR via Tess4J · Swagger / OpenAPI · JUnit 5 · Mockito · Postman · React 18 & Vite frontend · Maven

key features

Customer registration and onboarding · section-by-section information capture · identification, employment, financial and banking details · KYC document management with OCR cross-checking · risk profiling questionnaire · FATCA / CRS tax declaration, PEP screening, consents and e-signature · beneficiaries and trusted contact · application submission and status tracking · operations and compliance review workflow · append-only audit log and reports · role-based access for four staff and customer roles

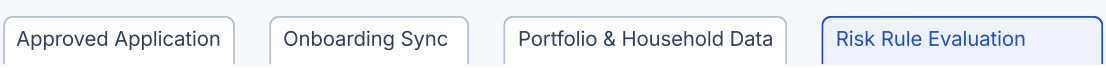
capstone_07 / perficient

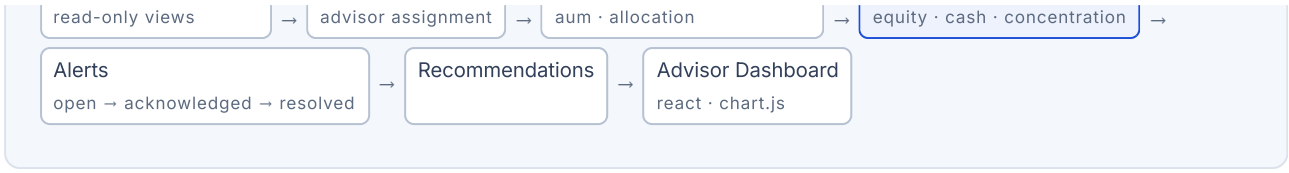
FINANCIAL TECHNOLOGY · DASHBOARD DEVELOPMENT · DATA VISUALIZATION · FASTAPI

Advisor360 — Wealth Management Portfolio & Household Dashboard

An advisor-facing dashboard that consolidates client relationships, household information, investment portfolios and financial risk indicators into one view. Approved onboarding applications flow into an advisor's book; the service then evaluates each portfolio against configurable risk rules, raises alerts, and derives recommendations an advisor can accept or dismiss.

architecture advisor_dashboard_flow





technology

Python 3.14 · FastAPI · SQLAlchemy 2 · Alembic · psycopg 3 · PyJWT · Pydantic Settings · Uvicorn · PostgreSQL (dedicated schema plus read-only views into the onboarding database) · React 18 · Vite · Chart.js 4 · pytest · Postman

key features

Advisor homepage with summary tiles and charts · client 360-degree overview · paged, filterable client list · household management with combined AUM · portfolio overview per client and across the whole book · asset allocation and sector analysis · Assets Under Management reporting · investment risk monitoring with a configurable rule engine · portfolio insights and financial recommendations · firm-level reports and an append-only audit trail · role scoping so an advisor sees only their own clients

risk indicators

Three portfolio risk-monitoring rules, each with a threshold held in the database: equity allocation above 80%, cash allocation above 25%, and single-stock concentration above 30%. Comparisons are exact decimal arithmetic, so 80.00% is not a breach and 80.01% is. These are monitoring indicators for an advisor to act on — not investment advice, and nothing is traded.

earlier_work / internship · research · hackathon

project_01 /

AI OCR FULL-STACK

NoshGuard: Smart Product Advisor

AI-powered advisor that reads packaged-food ingredient lists from a photo, evaluates them against a personal dietary profile, and returns an explainable decision with safer alternatives.

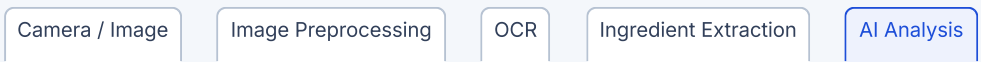
problem

Ingredient lists are dense, unfamiliar, and rarely matched to the reader's own dietary constraints. Generic food sites ignore personal allergens, conditions, or dietary choices, and few tools offer an explanation for why a product is (or isn't) suitable.

approach

Photograph the ingredient list, extract the text with OCR, structure it, then evaluate each ingredient against a user-defined health and dietary profile using a large language model producing a clear Safe / Caution / Avoid verdict with reasons and suggested alternatives.

architecture processing_pipeline





technology

PHP · Tailwind CSS · Python · Flask · Tesseract OCR · Mistral AI · MySQL

features

User registration · personalized dietary profile · ingredient scanning · OCR processing · AI analysis · product classification · explainable verdicts · alternative recommendations · decision history persisted to MySQL.

contribution

Implemented the end-to-end pipeline image handling into Tesseract OCR, ingredient structuring, the Mistral AI analysis layer, and the personalized classification (Safe / Caution / Avoid) with explanation generation. Built the evaluation and alternative-recommendation logic and wired the decision history into the database layer.

outcome

A working end-to-end prototype that turns an unreadable photo of an ingredient list into a clear, explainable, personal recommendation. Demonstrates a production-shaped OCR → LLM flow rather than a loose chain of scripts.

project_02

AI SPEECH-PROCESSING RESEARCH

Adaptive Emotion-Aware Speech Therapy Framework for Dysarthria

An AI-based research framework for adaptive speech therapy. It analyses a speaker's speech characteristics, emotional state, accent variation, and severity level then generates therapy exercises that adapt to the speaker rather than applying one fixed routine.

problem

Dysarthria affects speech clarity and intelligibility, usually with individualized impairments. Conventional therapy routines are generic and rarely account for the speaker's current emotional state, accent, or severity reducing both engagement and effectiveness.

approach

A speech-driven loop: analyze the patient's input, estimate characteristics and severity, detect emotion and accent influence, then adapt therapy content and difficulty. Feedback closes the loop and re-adjusts the next session.

research_pipeline speech_therapy_loop



technology

Python · speech processing · machine learning / deep learning · emotion recognition · accent-aware feature design

contribution

Designed the end-to-end framework and its adaptive therapy decision layer mapping speech and emotion analysis outputs into severity-appropriate, personalized exercise plans. Transitioned a conventional speech-therapy idea into a measured, data-driven research system.

project_03

FULL-STACK

Hospital Appointment & Surgery Information Platform

A patient-facing platform for clinic discovery, appointment booking, surgery information, downloadable PDF summaries, and embedded maps. Built during a Full Stack Developer internship.

Clinic Search → Booking → Surgery Info → PDF + Maps

angular · node.js · mongodb

project_04

GENERATIVE-AI HACKATHON

Text-to-Image Generation System

A hackathon-built generative AI prototype that turns text or spoken input into a generated image through a clean chatbot-style frontend, making the AI interaction approachable and immediate.

Text / Speech → Processing → AI Model → Generated Image

generative ai · speech recognition · ui/ux

contribution frontend ui/ux, chatbot interface, speech input

project_05

SYSTEMS ML

Log-Based Predictive Autoscaler

A predictive autoscaling system that consumes system and application logs, extracts behavioral features, and forecasts resource demand so infrastructure is adjusted before saturation, not after.

Logs → Feature Extraction → Prediction Model → Resource Demand → Autoscaling Decision

python · machine learning · systems

capstone_04 / full-stack development · e-commerce

PawPantry — Full Stack Pet Supplies E-commerce Platform

overview

PawPantry is a full-stack e-commerce application built as a Perficient capstone. It joins two independently developed halves — a React storefront and a Spring Boot REST API — into one working store. Shoppers browse and search the catalogue, fill a cart, check out, track orders and subscribe to repeat deliveries; store managers run the catalogue and orders from an admin area. The server owns every decision that matters: prices, stock, roles and order state.

full-stack integration

The browser only ever talks to the Vite dev server, which forwards `/api/*` to Spring Boot. That gives one origin, no CORS, and a strict same-site refresh cookie. The frontend consumes the API through RTK Query; the backend exposes it under an OpenAPI 3 contract with a matching Postman collection, so both halves are developed against the same documented interface. Shared test vectors keep cart pricing identical on both sides of the wire.

- Stateless JWT sign-in: a 15-minute access token plus a hashed, rotating 7-day refresh cookie, BCrypt password hashing, account lockout and rate limits.
- Server-side pricing only — per-category GST shown line by line and free shipping above a threshold — so the client can never set a price.
- Idempotent order creation, ETags and optimistic locking on admin edits, a payment reconciliation job, an outbox and server-sent events for live order updates in every open tab.
- PostgreSQL schema owned by Flyway migrations (V1–V17); the API is validated against the schema rather than generating it.

technology stack

frontend	React 19 · Vite 8 · React Router 6 · Redux Toolkit 2 with RTK Query · JavaScript
backend	Java 21 · Spring Boot 3.5 · Spring MVC · Spring Data JPA · Spring Security · REST APIs · JWT authentication
database	PostgreSQL with Flyway migrations
testing	JUnit 5 · MockMvc · ArchUnit · JaCoCo · Vitest · Testing Library · Playwright with axe accessibility checks
tools	Postman · Swagger / OpenAPI 3 · Git & GitHub · Maven wrapper · ESLint · Prettier

key features

- User registration, sign-in and account management
- Server-side search, category / brand / price / rating filters, sorting and paging
- Shopping cart and wishlist, including guest carts merged on sign-in
- Order management and tracking with a status timeline, cancel window and PDF invoice
- Admin orders, sales report, audit log and CSV import
- Product catalogue with admin product management
- Product reviews and ratings with a verified-purchase badge
- Three-step checkout with an idempotency key
- Subscribe & Save: pause, resume, skip, change or cancel a repeat delivery
- Natural-language catalogue search whose filters are validated server-side

honest status

> Payment processing is implemented as Cash on Delivery plus a mock card gateway, with every outcome handled explicitly (approved, declined, timeout, gateway failure, answer lost). **No live third-party payment provider is integrated.** All modules are built and tested, but milestones are marked "built" rather than signed off: several manual browser checklists are outstanding, the database-dependent tests have so far run against mocks, and browser testing has been Chromium-only.

capstone_06 / enterprise application · fintech

Wealth Management — New Account Opening Platform

overview

An enterprise-oriented financial services application for digital account onboarding in wealth management. A client registers, completes an application made up of fifteen information sections, uploads identity and KYC documents, answers a risk-profiling questionnaire, signs the regulatory declarations and submits. Operations and Compliance staff then review the submission, verify the documents, record remarks and approve, return or reject it. Administrators manage staff accounts and read the audit trail and aggregate reports.

architecture

A modular monolith: one Spring Boot application and one PostgreSQL database, with packages organised by business feature and the same controller → service → repository → entity layering inside each. The API is REST/JSON under `/api/v1`, stateless, documented with OpenAPI, and every error passes through one global handler into a standard response envelope. The database schema is owned by Flyway (V1 through V22) and the application validates against it rather than generating it. Uploaded documents go to a local file system behind a storage interface, and sensitive fields such as bank account numbers are encrypted at the field level.

business functionality implemented

- Customer registration, login and profile
- Personal information and residential address capture
- Identification details and employment information
- Financial information and banking details
- Investment profile and risk-profiling questionnaire
- FATCA / CRS tax declaration and PEP screening
- Regulatory consents and e-signature
- Beneficiary management and trusted contact
- Document upload by category, with OCR extraction used to cross-check the details on the document
- KYC verification by Operations
- Application submission, with edits locked afterwards
- Application status tracking and immutable status history
- Operations and compliance review with reviewer remarks and approve / return / reject decisions
- Append-only audit log, aggregate reports and admin staff-account management

technology stack

backend	Java 25 · Spring Boot 4.1 · Spring MVC · Spring Data JPA / Hibernate · Maven
security	Spring Security as an OAuth2 resource server · JWT access tokens with rotating refresh tokens · BCrypt · role-based access for CUSTOMER, OPERATIONS, COMPLIANCE and ADMIN, plus ownership checks
database	PostgreSQL with Flyway migrations (V1–V22)
documents	Local file-system storage behind a storage interface · Tesseract OCR via Tess4J for document extraction
frontend	React 18 · Vite — customer application wizard and staff review screens
testing	JUnit 5 · Mockito · Maven Failsafe integration tests · Postman collections, one per milestone

verification

> At the last full run: 198 unit tests and 276 integration tests on the backend, plus twelve Postman collections covering the milestones end to end. Tokens are signed with a shared HS256 secret, which is appropriate for a proof of concept rather than production. **No regulatory compliance is claimed** and no external KYC/AML vendor is integrated — the KYC and screening steps are internal workflow, not third-party verification.

capstone_07 / fintech · dashboard · data visualization

Advisor360 — Wealth Management Portfolio & Household Dashboard

overview

Advisor360 is the second half of the same wealth-management product: once a customer's account application is approved, an advisor serves them. It gives the advisor a consolidated view of their book — clients, households, portfolios, assets under management and risk indicators — on one dashboard, and continues from monitoring into concrete, explained suggestions.

architecture

A separate Python service alongside the onboarding backend. It owns its own PostgreSQL schema, migrated with Alembic, and reads the onboarding side through exactly two read-only database views — so client onboarding data is never duplicated or written from here. Both services accept the same access token, so one sign-in covers the whole product, and the React website routes advisor paths to this service and everything else to the onboarding API. Twenty-six endpoints, a shared response envelope, request ids on every response, and paging on every list.

business functionality implemented

- Advisor homepage with summary tiles, charts and clients needing attention
- Paged client list with search and filters by risk profile, household and alert state
- Portfolio overview per client and across the whole book
- Assets Under Management reporting
- Alert lifecycle: open → acknowledged → resolved, with a real breach reappearing at the next check
- Onboarding hand-off: approved applications pulled into advisors' books, with an assignment suggestion and the data behind it
- Client 360-degree overview: profile, risk profile, AUM, accounts, alerts and recommendations
- Household management with members and combined AUM
- Asset allocation, sector and account-type analysis with top holdings
- Investment risk monitoring with a database-driven rule engine
- Portfolio insights and financial recommendations an advisor can accept or dismiss
- Firm-level reports, append-only audit trail, and client reassignment for administrators

portfolio risk indicators

- **High equity** — equity allocation above 80% of the portfolio.
- **Idle cash** — cash allocation above 25% of the portfolio.
- **Concentration** — a single stock above 30% of the portfolio.

Each threshold is stored in the database so it can change without a code change, and comparisons use exact decimal arithmetic rather than floating point: 80.00% is not a breach, 80.01% is. A breach raises an alert, and each alert type maps to one recommendation type — reduce equity, deploy cash or diversify — carrying the measured percentage, the limit, the target and a written rationale.

> These are **risk-monitoring indicators for a qualified advisor to act on, not investment advice**. Accepting a recommendation records the advisor's decision; **nothing is traded**. The targets and margins behind the suggested amounts are placeholders awaiting business sign-off, and no AI model is called — the optional narration is a local template.

technology stack

api	Python 3.14 · FastAPI · Uvicorn · Pydantic Settings · PyJWT
data	SQLAlchemy 2 · Alembic migrations · psycopg 3 · PostgreSQL (own schema plus read-only views into the onboarding schema)
frontend	React 18 · Vite · Chart.js 4 — dashboard tiles, allocation rings and portfolio charts
testing	pytest (628 tests at the last full run) · seven Postman collections · security-matrix tests covering every endpoint and role

/04 / Professional Experience

2026 – present current

Associate Technical Consultant

Perficient Inc.

Product Development (Custom Delivery) · Bengaluru, India

Currently working as an Associate Technical Consultant at Perficient, contributing to technical learning and software development initiatives within the Product Development (Custom Delivery) business area. Work to date has centred on building three substantial capstone applications — a full-stack e-commerce platform, an enterprise account-onboarding system and an advisor portfolio dashboard — covering frontend, backend, API and database layers end to end.

technical areas

Software Development | Full Stack Application Development | Frontend & Backend Technologies | API Development & Integration | Database Management | Debugging & Testing | Collaborative Development Workflows | Git & GitHub

Jun 2025 - Jul 2025

Full Stack Developer Intern

Kekkar Infotech & Services

Developed a hospital appointment & surgery information platform using Angular, Node.js, and MongoDB featuring clinic search, appointment booking, surgery information, downloadable PDF generation, and embedded maps integration.

/05 / Research

Adaptive Emotion-Aware Speech Therapy Framework for Dysarthria Using AI

problem

Conventional dysarthria therapy applies fixed exercise routines, ignoring each speaker's unique impairment profile severity, accent, and emotional state during a session.

motivation

Therapy is more effective when it adapts. A framework that continuously reads the speaker's speech and emotion signals can keep exercises challenging but achievable, and adjust when the speaker is struggling.

methodology

Speech input is preprocessed and reduced to acoustic features. Models classify speech characteristics, emotion, and severity; the assessment feeds an adaptive therapy generator that selects and stages exercises; feedback updates the next round.

ai / ml approach

Machine and deep learning classifiers over extracted speech features for emotion and severity estimation, with accent-aware feature handling, and a rule-informed therapy decision layer that converts model output into staged exercise plans.

dataset context

Developed as a research framework, evaluation targets standard dysarthric and emotion speech corpora in the domain.

contribution

Defined the adaptive therapy decision layer that connects speech/emotion analysis to severity-appropriate exercise generation turning a one-size-fits-all therapy model into a measurable, feedback-driven system.

```
// therapy_loop

01  input speech
02  preprocess · features
03  speech & emotion analysis
04  severity assessment
05  adaptive therapy
```

```
06  feedback → loop
```

/06 / Education

2022 – 2026

Bachelor of Technology — Computer Science & Engineering

Amrita Vishwa Vidyapeetham, Bengaluru

specialization: computer science & engineering

2021 – 2022

12th Grade

specialization: PCMC

Sri Chaitanya Techno, School

/07 / Certifications

✓ Python Programming & Data Analytics — Udemy

/08 / Hackathons & Achievements

NNDL Hackathon

Built a text-to-image generation system as part of the NNDL Hackathon team. Focus areas included:

- text-to-image generation
- frontend / ui development – chatbot-style interface
- ai interaction design
- speech recognition integration

/10 / Contact

Let's build something useful.

Open to conversations about software engineering, full stack development, enterprise applications and applied AI. The quickest way to reach me is email.

Nishit.kumar@perficient.com

in linkedin.com/in/nishit-kumar

</> github.com/Nishitkumar25